Implementação de PKI com HSM

Autor:
Leonardo Oliveira Feitosa

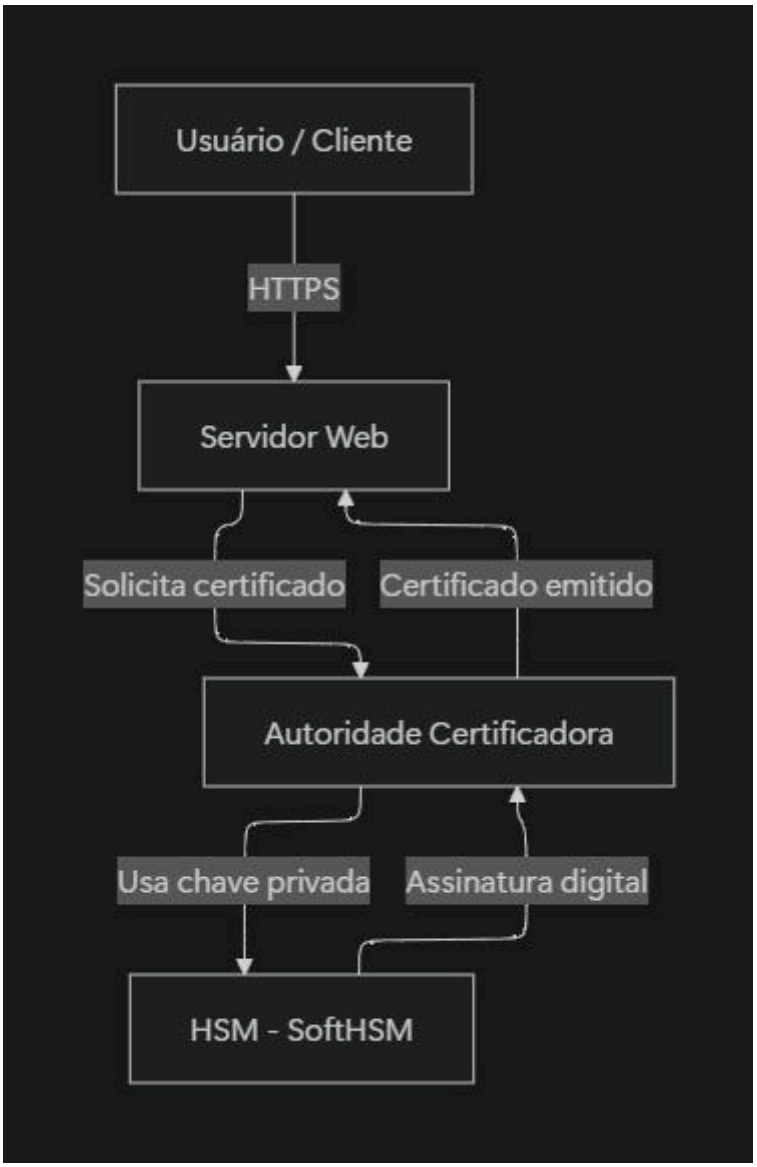
2026

1. Introdução:

Este laboratório tem como objetivo demonstrar, na prática, a implementação dos principais conceitos de Infraestrutura de Chaves Públicas (PKI) e gerenciamento de certificados digitais em um ambiente controlado. Durante as etapas do experimento, foi realizada a criação de uma Autoridade Certificadora (CA) utilizando o OpenSSL, com a chave privada protegida por um Hardware Security Module (HSM) simulado através do SoftHSM utilizando o padrão PKCS#11.

A partir dessa estrutura, foram gerados e assinados certificados digitais para um servidor web, possibilitando a configuração de comunicação segura via HTTPS em um servidor Apache HTTP Server. O laboratório busca, portanto, integrar conceitos de criptografia aplicada, gerenciamento seguro de chaves e certificados, demonstrando como essas tecnologias são utilizadas em ambientes corporativos para garantir autenticidade, confidencialidade e integridade nas comunicações de rede.

2. Arquitetura do Laboratório:



3. Preparação do Ambiente:

```shell
sudo apt update && apt upgrade -y
```

Atualizar o sistema.

```shell
sudo apt install -y \
softhsm2 \
opensc \
libengine-pkcs11-openssl \
openssl \
apache2 \
```

Instalar os serviços e ferramentas para o laboratório.

4. HSM:

4.1 - Inicializar o token HSM (inserir o PIN).

```shell
softhsm2-util --init-token \
--free \
--label "PKI-HSM"
```

```shell
softhsm2-util --show-slots
```

Verificar o token criado.

4.2 Gerar chave da CA dentro do HSM.

```shell
pkcs11-tool \
--module /usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so \
--login \
--pin 1234 \
--keypairgen \
--key-type rsa:2048 \
--id 01 \
--label CAkey
```

```shell
pkcs11-tool \
--module /usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so \
--login \
--pin 1234 \
--list-objects
```

Listar a chave.

4.3 Gerar CSR usando a chave do HSM.

```shell
export PKCS11_MODULE_PATH=/usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so
export PKCS11_PIN=1234

openssl req \
-engine pkcs11 \
-keyform engine \
-new \
```

```
-key "pkcs11:object=CAkey;type=private" \
-out ca.csr \
-config ca.cnf
```

4.4 Criar certificado da Root CA.

```
Shell
openssl x509 \
-req \
-in ca.csr \
-engine pkcs11 \
-keyform engine \
-signkey "pkcs11:object=CAkey;type=private" \
-extfile ca-ext.cnf \
-days 3650 \
-out ca.crt
```

```
Shell
openssl x509 -in ca.crt -text -noout
```

Verificar certificado.

4.5 Criar CSR do servidor.

```
Shell
openssl req -new \
-newkey rsa:2048 \
-nodes \
-keyout server.key \
-out server.csr
```

CN = lab-server.

4.6 Assinar certificado do servidor com a CA.

```
Shell
openssl ca \
```

```
-config openssl-ca.cnf \
-engine pkcs11 \
-keyform engine \
-in server.csr \
-out server.crt \
-days 365
```

```
Shell
openssl verify -CAfile ca.crt server.crt
```

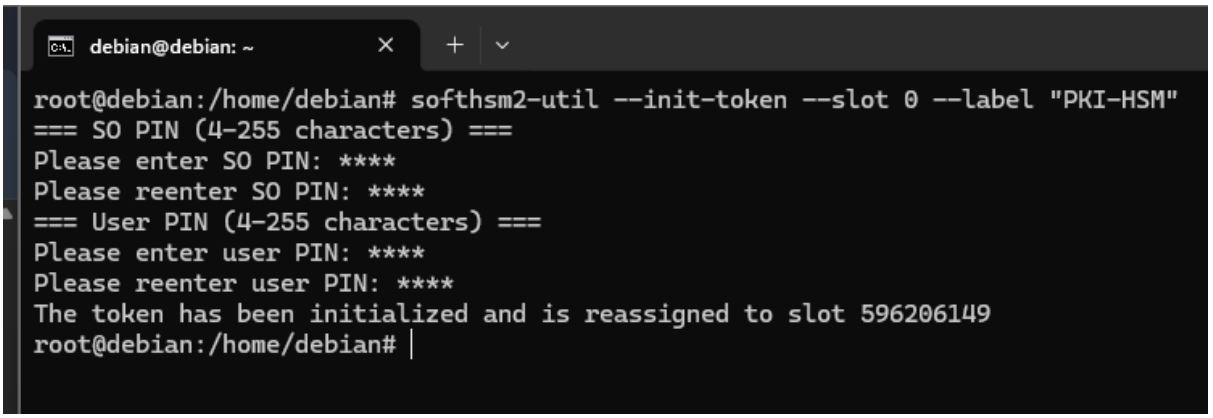
Verificar certificado.

4.7 Instalar CA no sistema.

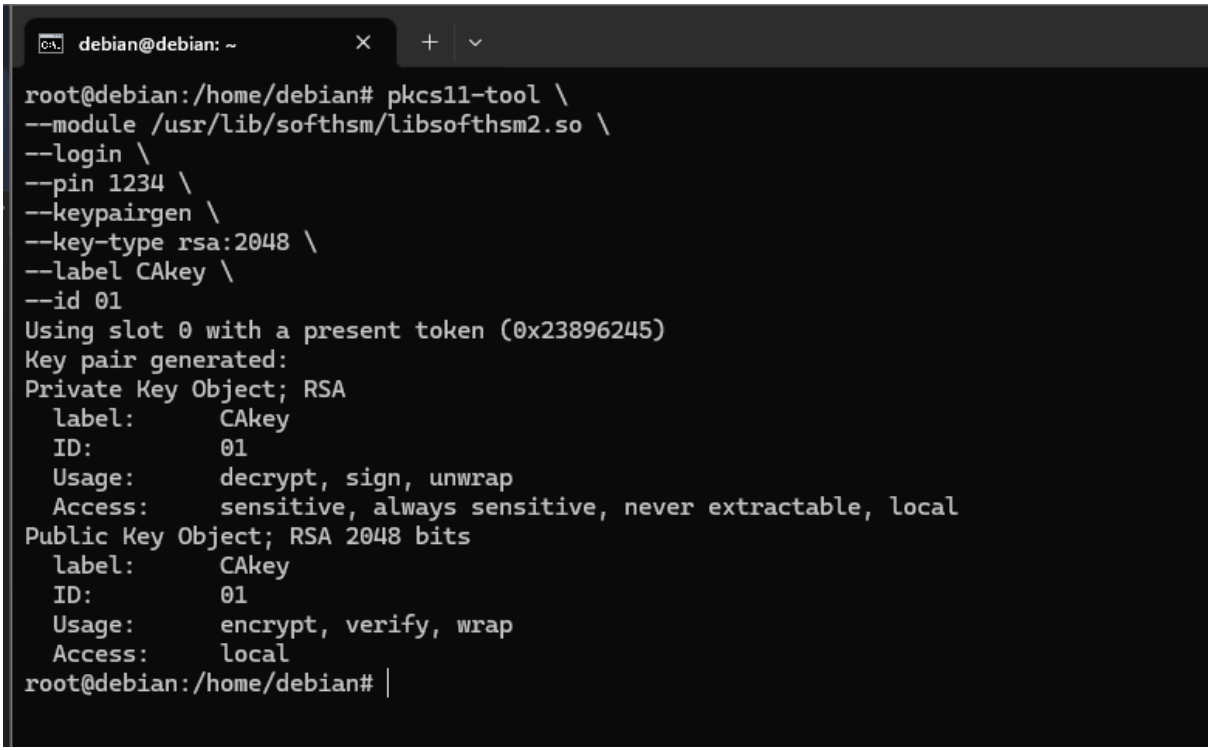
```
Shell
sudo cp ca.crt /usr/local/share/ca-certificates/lab-root-ca.crt
sudo update-ca-certificates
```

Evidências:

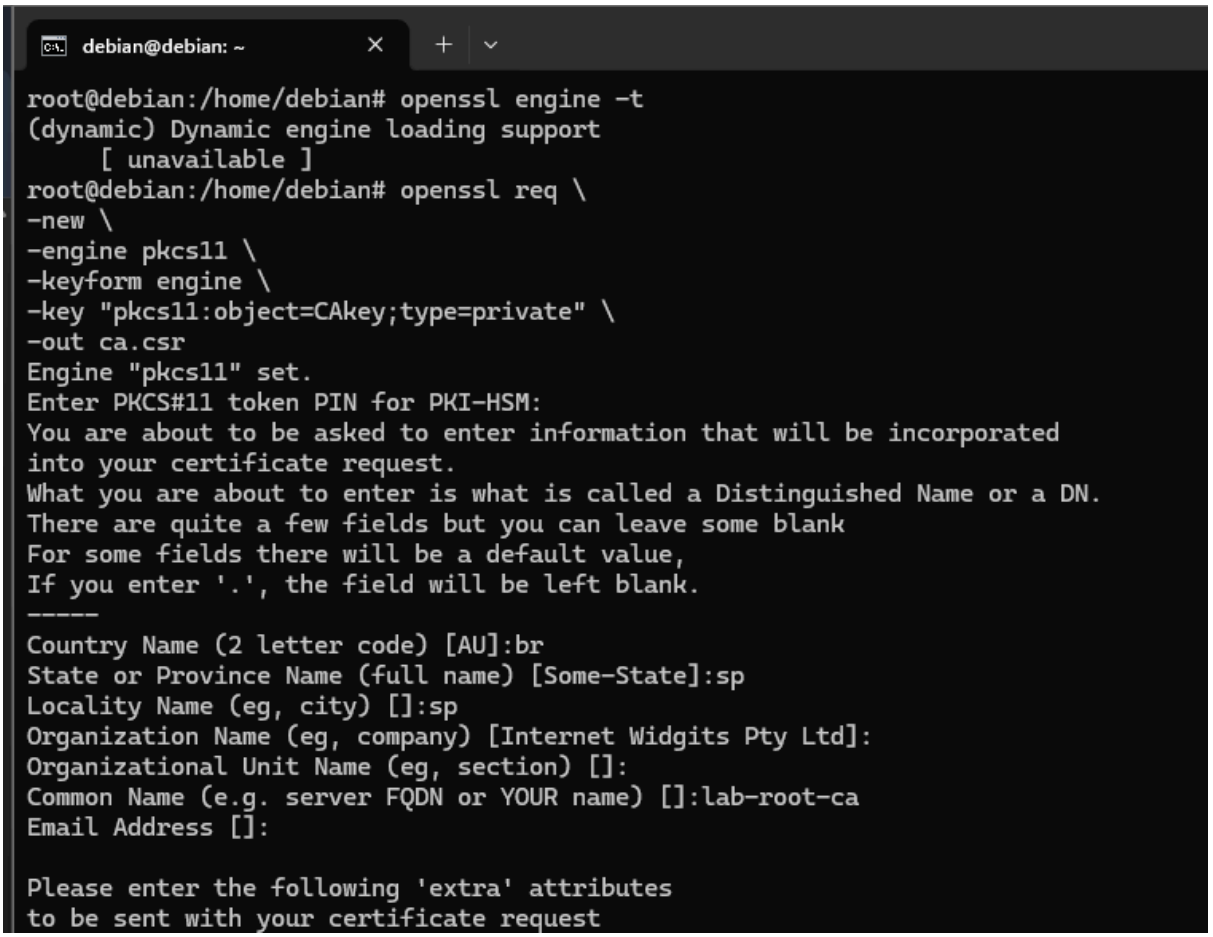
4.1



4.2

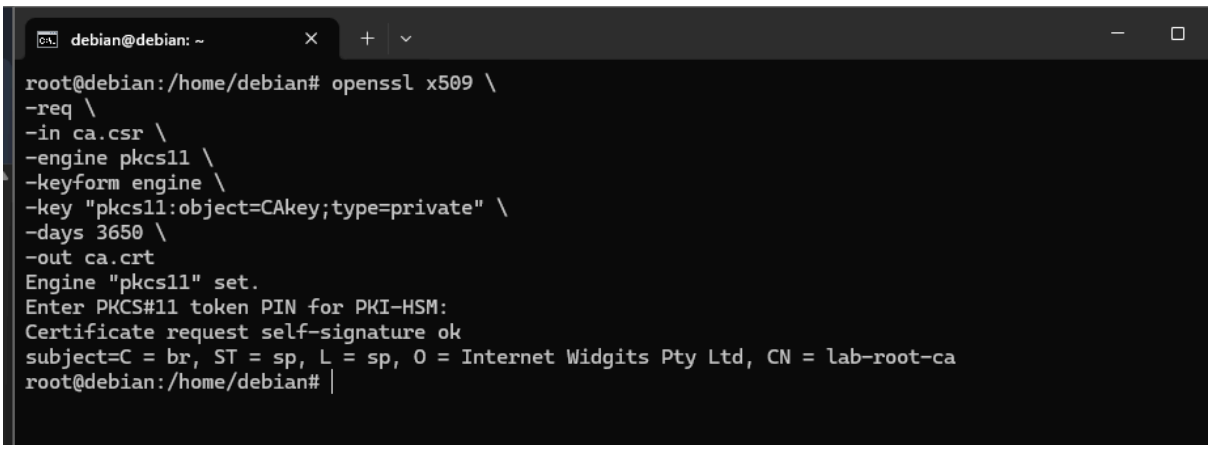


4.3

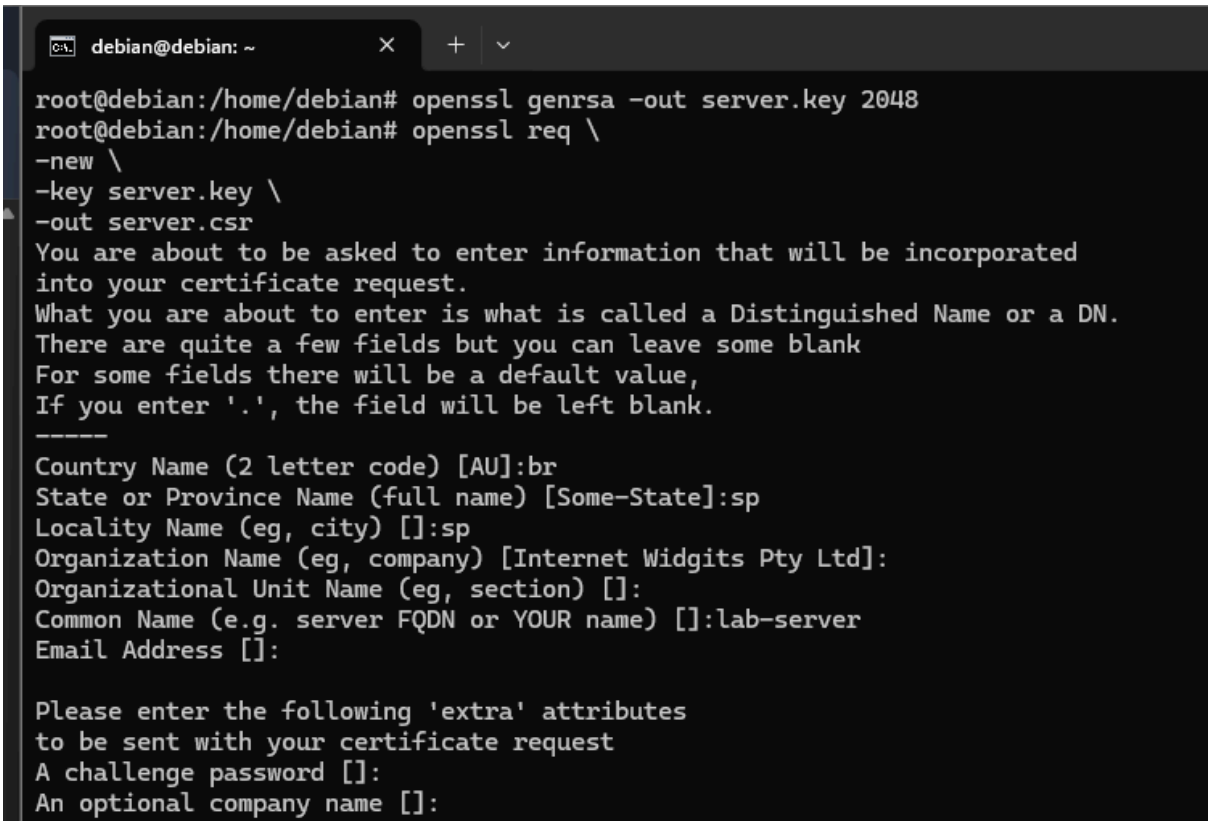

debian@debian: ~
root@debian:/home/debian# openssl engine -t
(dynamic) Dynamic engine loading support
[unavailable]
root@debian:/home/debian# openssl req \
-new \
-engine pkcs11 \
-keyform engine \
-key "pkcs11:object=CAkey;type=private" \
-out ca.csr
Engine "pkcs11" set.
Enter PKCS#11 token PIN for PKI-HSM:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

Country Name (2 letter code) [AU]:br
State or Province Name (full name) [Some-State]:sp
Locality Name (eg, city) []:sp
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:lab-root-ca
Email Address []:
Please enter the following 'extra' attributes
to be sent with your certificate request

4.4

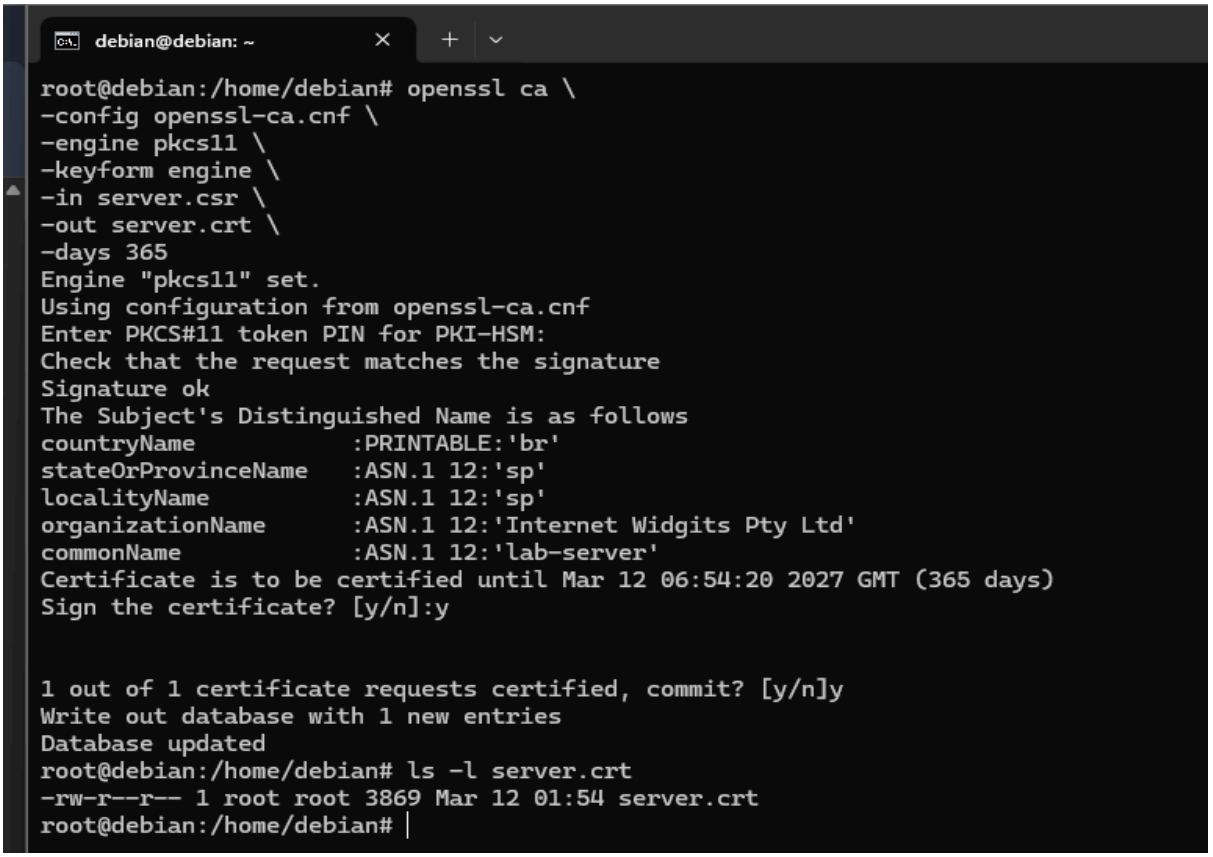

debian@debian: ~
root@debian:/home/debian# openssl x509 \
-req \
-in ca.csr \
-engine pkcs11 \
-keyform engine \
-key "pkcs11:object=CAkey;type=private" \
-days 3650 \
-out ca.crt
Engine "pkcs11" set.
Enter PKCS#11 token PIN for PKI-HSM:
Certificate request self-signature ok
subject=C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
root@debian:/home/debian#

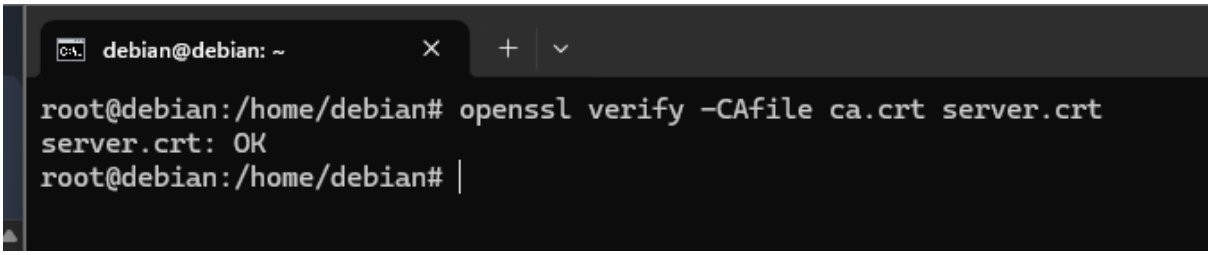
4.5


debian@debian: ~
root@debian:/home/debian# openssl genrsa -out server.key 2048
root@debian:/home/debian# openssl req \
-new \
-key server.key \
-out server.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

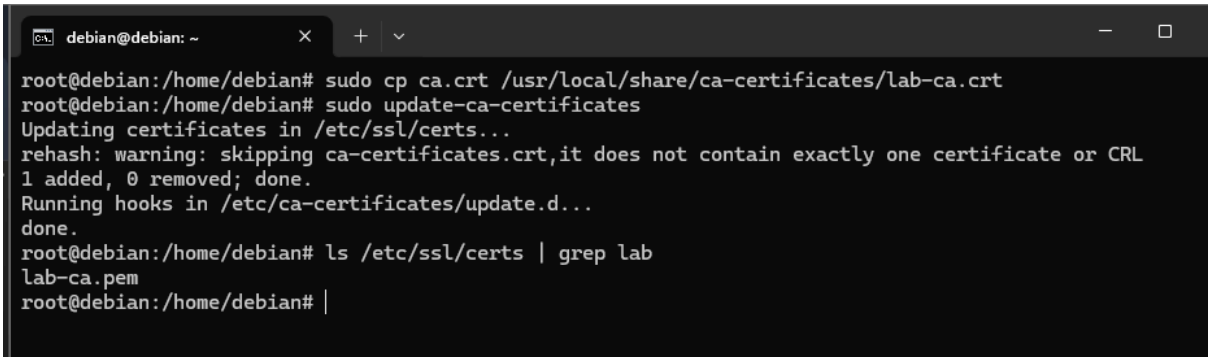
Country Name (2 letter code) [AU]:br
State or Province Name (full name) [Some-State]:sp
Locality Name (eg, city) []:sp
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:lab-server
Email Address []:
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:

4.6


debian@debian: ~
root@debian:/home/debian# openssl ca \
-config openssl-ca.cnf \
-engine pkcs11 \
-keyform engine \
-in server.csr \
-out server.crt \
-days 365
Engine "pkcs11" set.
Using configuration from openssl-ca.cnf
Enter PKCS#11 token PIN for PKI-HSM:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName :PRINTABLE:'br'
stateOrProvinceName :ASN.1 12:'sp'
localityName :ASN.1 12:'sp'
organizationName :ASN.1 12:'Internet Widgits Pty Ltd'
commonName :ASN.1 12:'lab-server'
Certificate is to be certified until Mar 12 06:54:20 2027 GMT (365 days)
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Database updated
root@debian:/home/debian# ls -l server.crt
-rw-r--r-- 1 root root 3869 Mar 12 01:54 server.crt
root@debian:/home/debian#


debian@debian: ~
root@debian:/home/debian# openssl verify -CAfile ca.crt server.crt
server.crt: OK
root@debian:/home/debian#

4.7


debian@debian: ~
root@debian:/home/debian# sudo cp ca.crt /usr/local/share/ca-certificates/lab-ca.crt
root@debian:/home/debian# sudo update-ca-certificates
Updating certificates in /etc/ssl/certs...
rehash: warning: skipping ca-certificates.crt,it does not contain exactly one certificate or CRL
1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
root@debian:/home/debian# ls /etc/ssl/certs | grep lab
lab-ca.pem
root@debian:/home/debian#

5. Apache com certificados:

Editar o arquivo de configuração em “/etc/apache2/sites-available/default-ssl.conf” e adicionar as informações abaixo:

```
Shell
SSLEngine on

SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key
SSLCertificateChainFile /etc/apache2/ssl/ca.crt
```

Criar um diretório de certificados e copiar os certificados para o diretório, ativar o SSL e reiniciar o serviço:

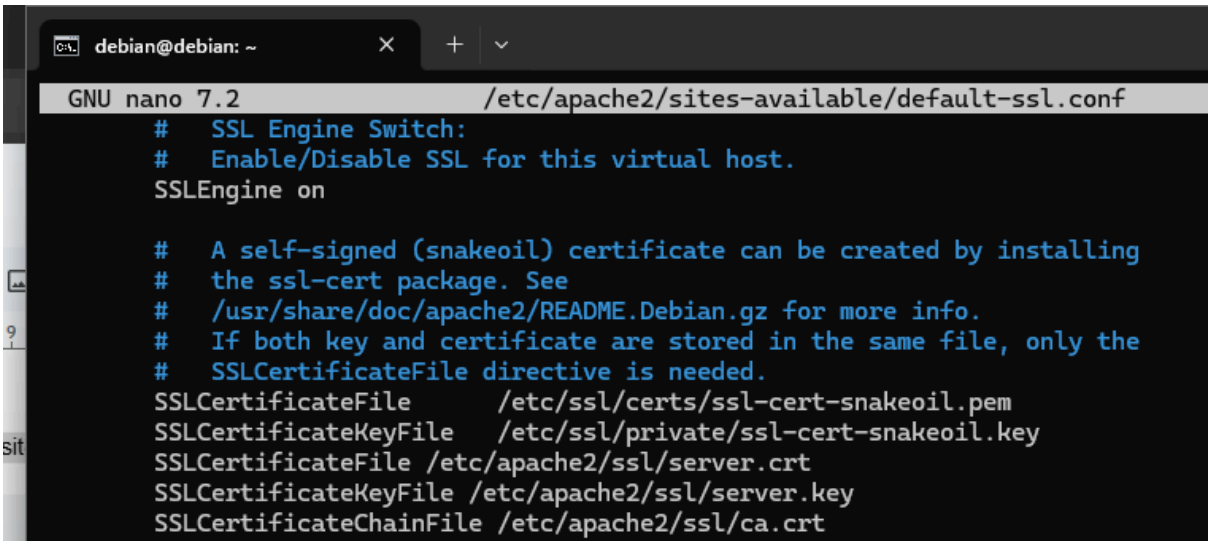
```
Shell
sudo mkdir /etc/apache2/ssl
sudo cp server.crt /etc/apache2/ssl/
sudo cp server.key /etc/apache2/ssl/

sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl restart apache2
```

Testar apache com os certificados:

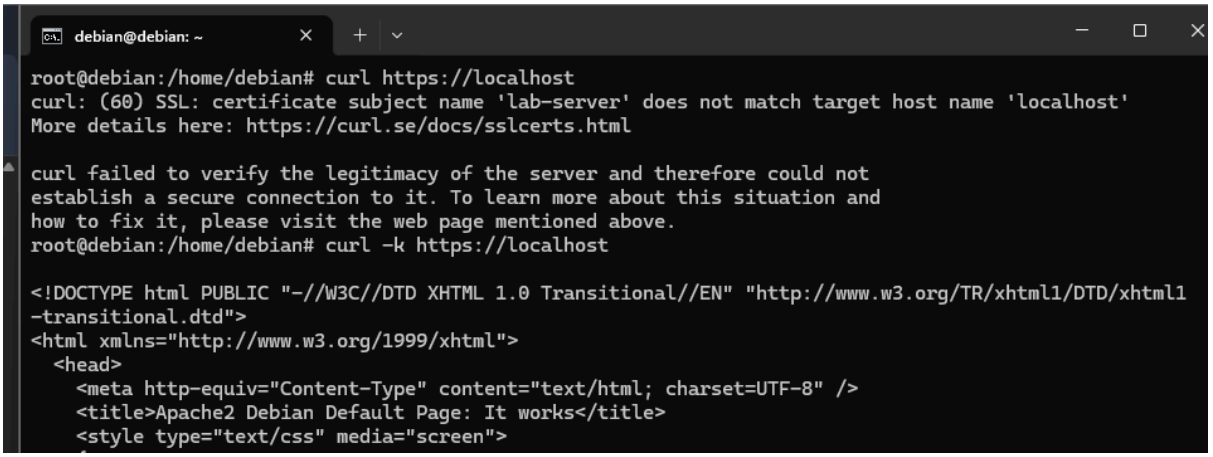
```
Shell
curl -k https://localhost

openssl s_client -connect localhost:443
```

Saída esperada, “Verify return code: 0 (ok)”.

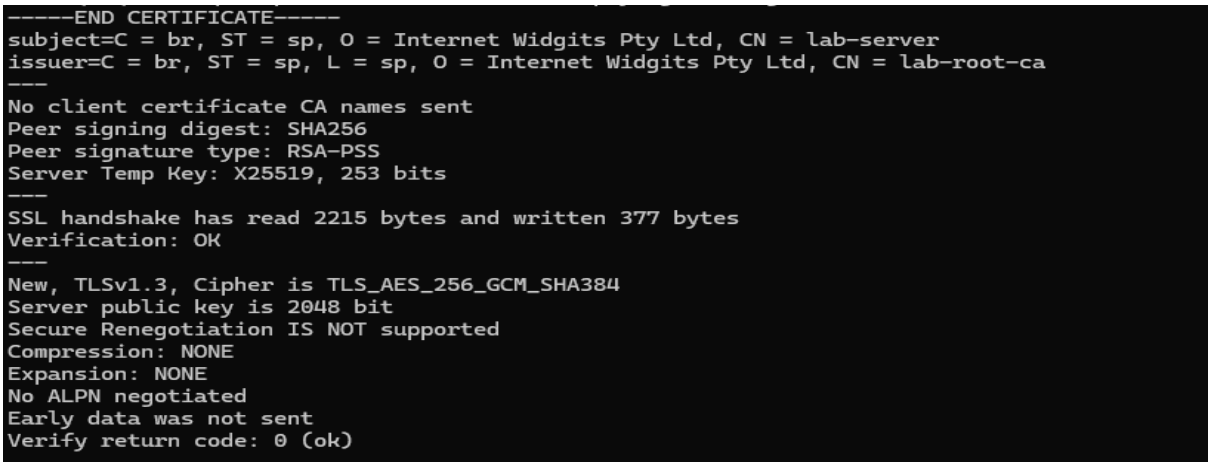
Evidências:

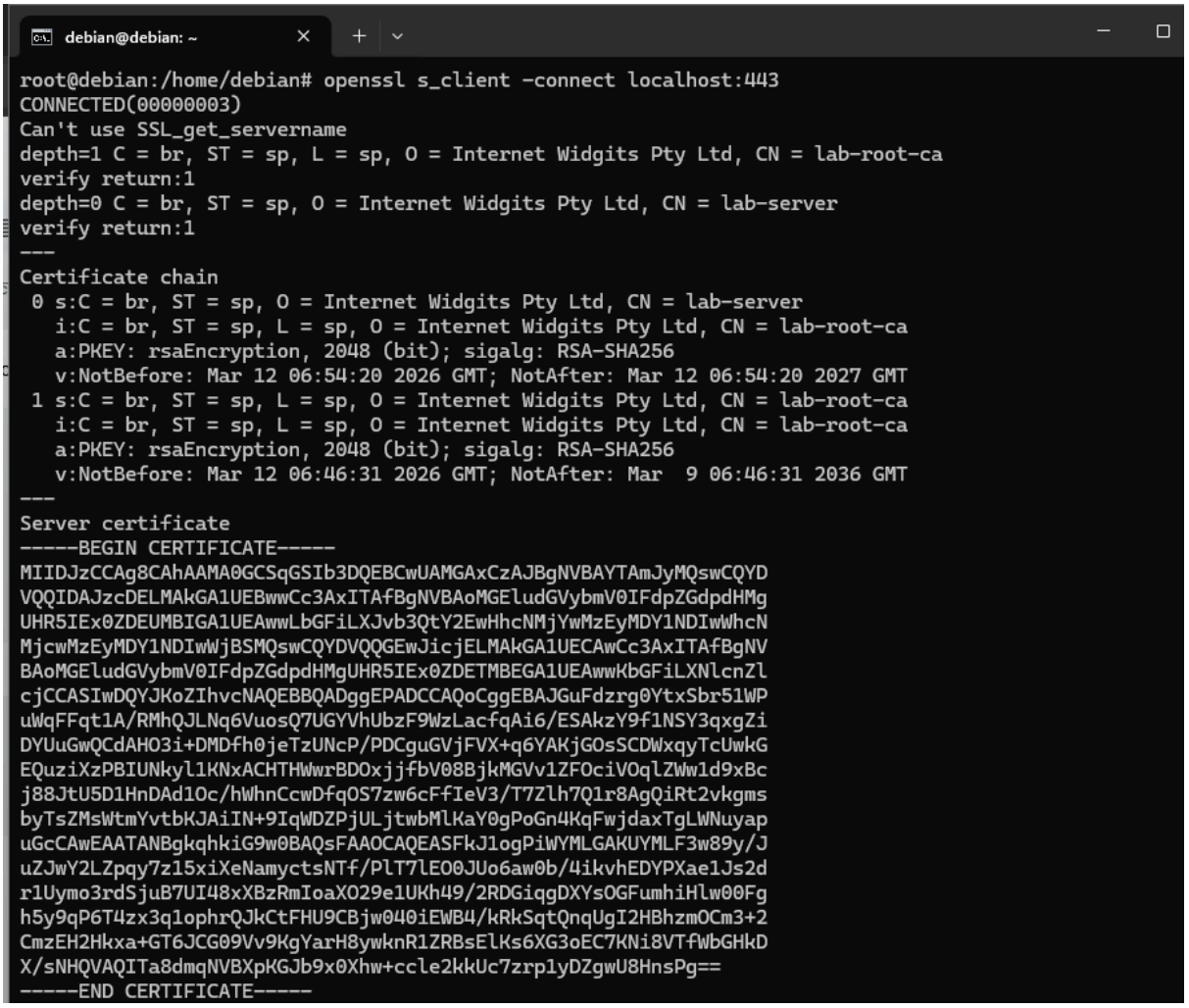


Arquivo de configuração.



Local host.





Validação do certificado no apache.

6. Vault como Intermediate CA:

O Vault está atuando como a autoridade certificadora intermediária, emitindo certificados TLS para aplicações. Ele automatiza a geração, distribuição e revogação de certificados usando a infraestrutura PKI criada, assim os serviços do lab podem obter certificados válidos sem precisar gerar ou assinar manualmente na CA.

```shell
vault server -dev
```

Abrir o servidor para pegar as credenciais.

```shell
export VAULT_ADDR='http://127.0.0.1:8200'

export VAULT_TOKEN="SEU_ROOT_TOKEN"
```

Logar.

```shell
vault secrets enable -path=pki_int pki

vault secrets tune -max-lease-ttl=43800h pki_int
```

Criando a PKI para a Intermediate CA.

```shell
vault write -format=json pki_int/intermediate/generate/internal \
common_name="Lab Intermediate CA" \
ttl=43800h > intermediate.json

jq -r '.data.csr' intermediate.json > intermediate.csr
```

Gerando e extraindo o CSR.

```shell
openssl ca \
-config openssl-ca.cnf \
-engine pkcs11 \
-keyform engine \
-in intermediate.csr \
-out intermediate.crt \
-days 1825 \
-extfile intermediate-ext.cnf
```

Assinar a Intermediate com a Root (HSM).

```shell
vault write pki_int/intermediate/set-signed \
certificate=@intermediate.crt
```

Importar a Intermediate no Vault.

```shell
vault write pki_int/config/urls \
issuing_certificates="http://127.0.0.1:8200/v1/pki_int/ca" \
crl_distribution_points="http://127.0.0.1:8200/v1/pki_int/crl"
```

Configurar URLs da PKI.

```shell
vault write pki_int/roles/web \
allowed_domains="lab.local" \
allow_subdomains=true \
max_ttl="72h"
```

Criar role para emissão de certificados.

```shell
vault write -format=json pki_int/issue/web \
common_name="app.lab.local" > cert.json
```

Emitir um certificado.

```shell
jq -r '.data.certificate' cert.json > app.crt

jq -r '.data.private_key' cert.json > app.key

jq -r '.data.issuing_ca' cert.json > ca_chain.crt
```

Extrair certificados.

```shell
openssl x509 -in app.crt -text -noout
```

Verificar o certificado.

```shell
SSLEngine on
```

```
SSLCertificateFile /home/debian/app.crt
SSLCertificateKeyFile /home/debian/app.key
SSLCertificateChainFile /home/debian/ca_chain.crt
```

Usar os certificados novos no apache e depois reiniciar o serviço.

Revogar um certificado:

```shell
openssl x509 -in app.crt -noout -serial
```

Obter o serial number do certificado.

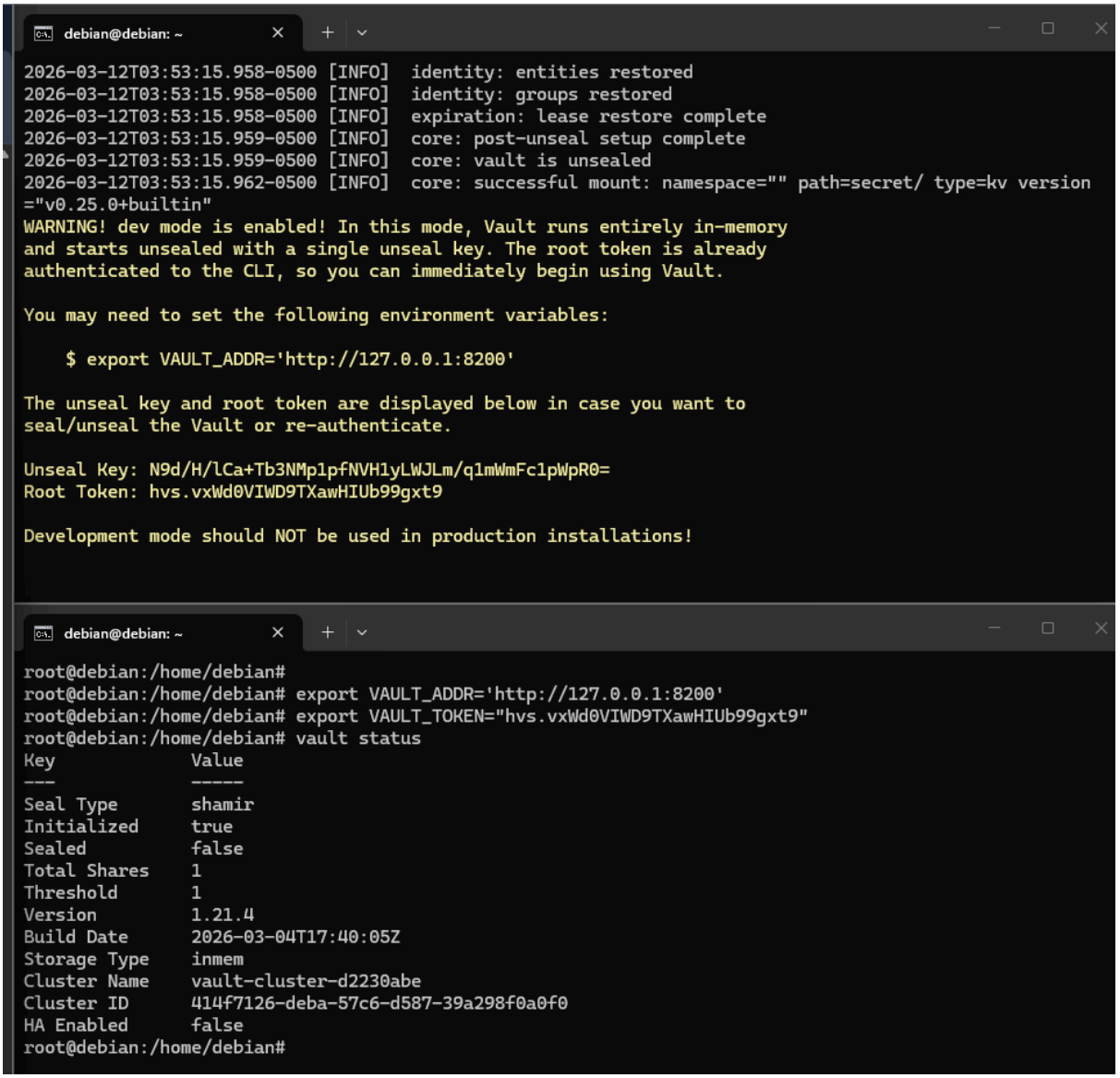
```shell
vault write pki_int/revoke serial_number="serial_number"
```

Revogando no vault, com isso o certificado deixa de ser confiavel.

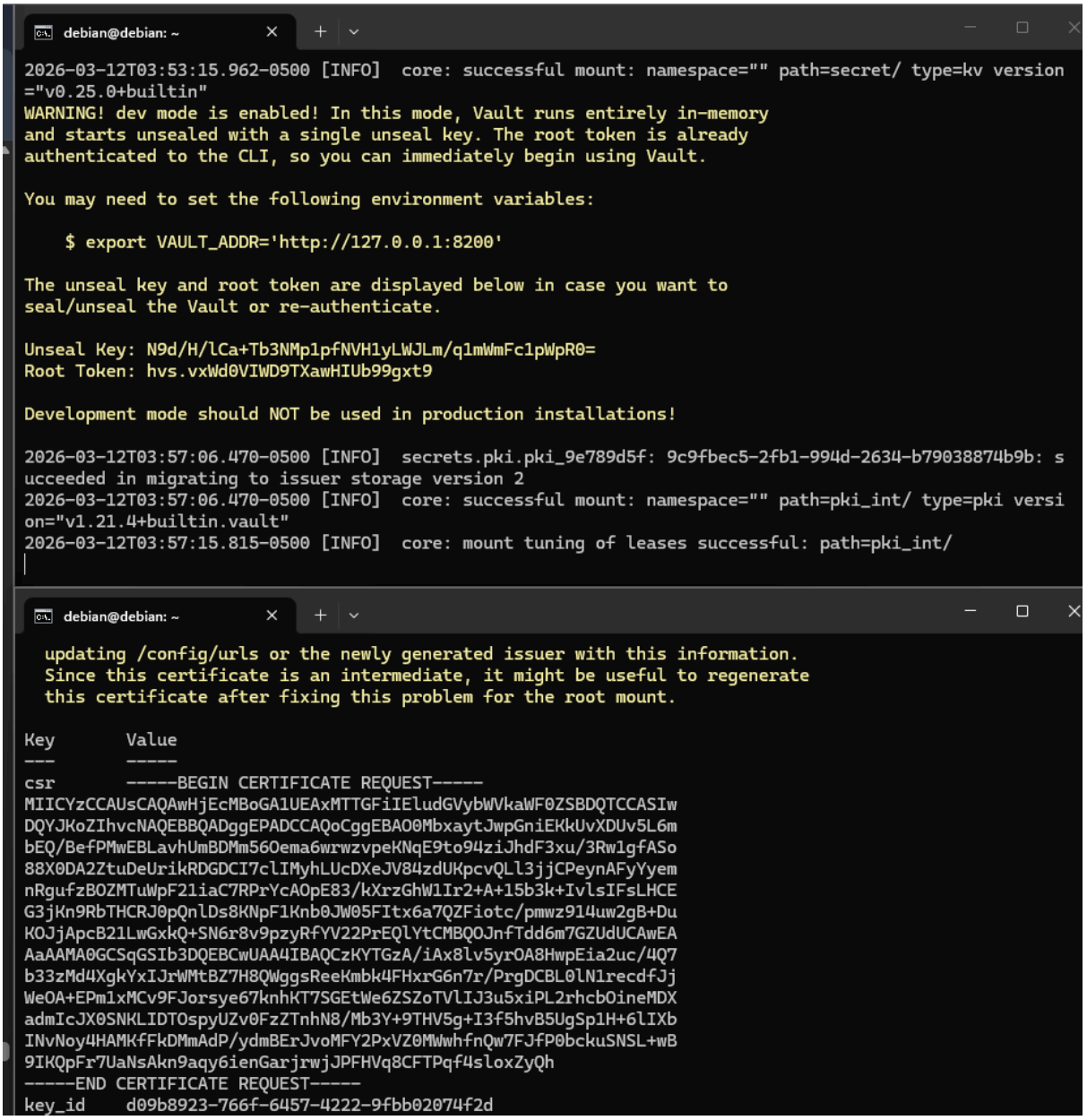
```shell
curl http://127.0.0.1:8200/v1/pki_int/crl -o crl.pem
```

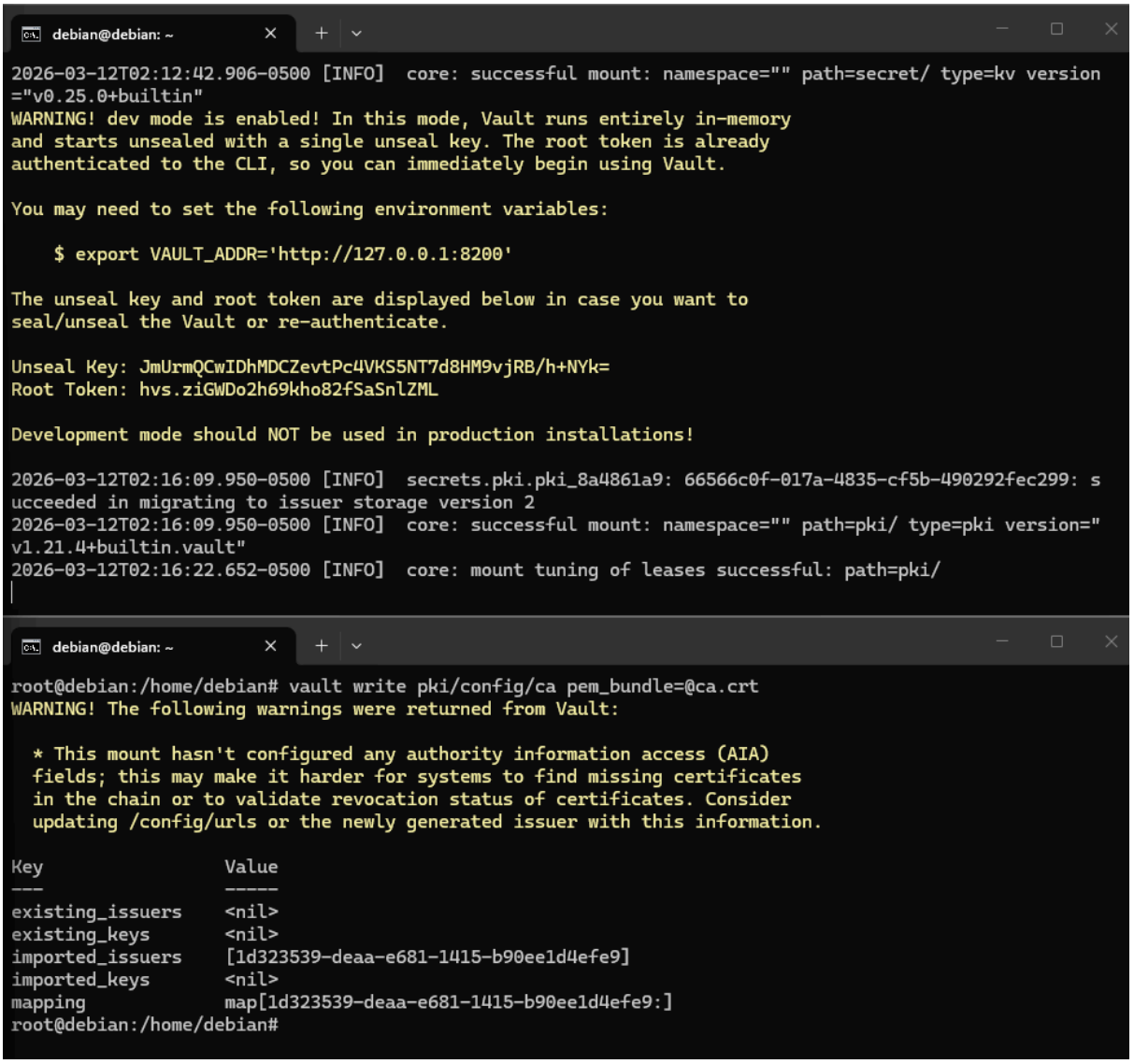
Baixar a lista de certificados revogados.

Evidências:

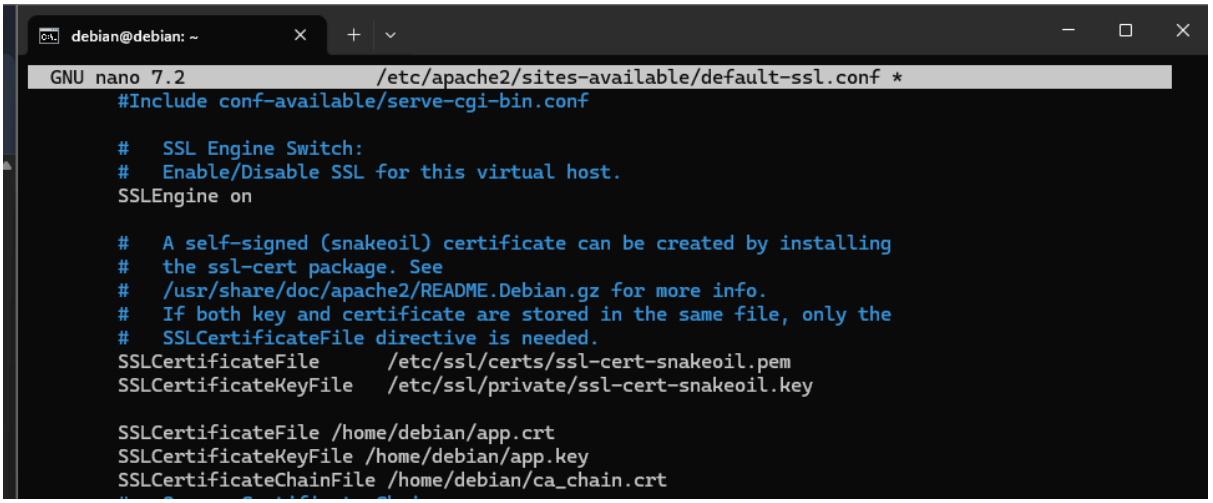


Configurando a intermediate.

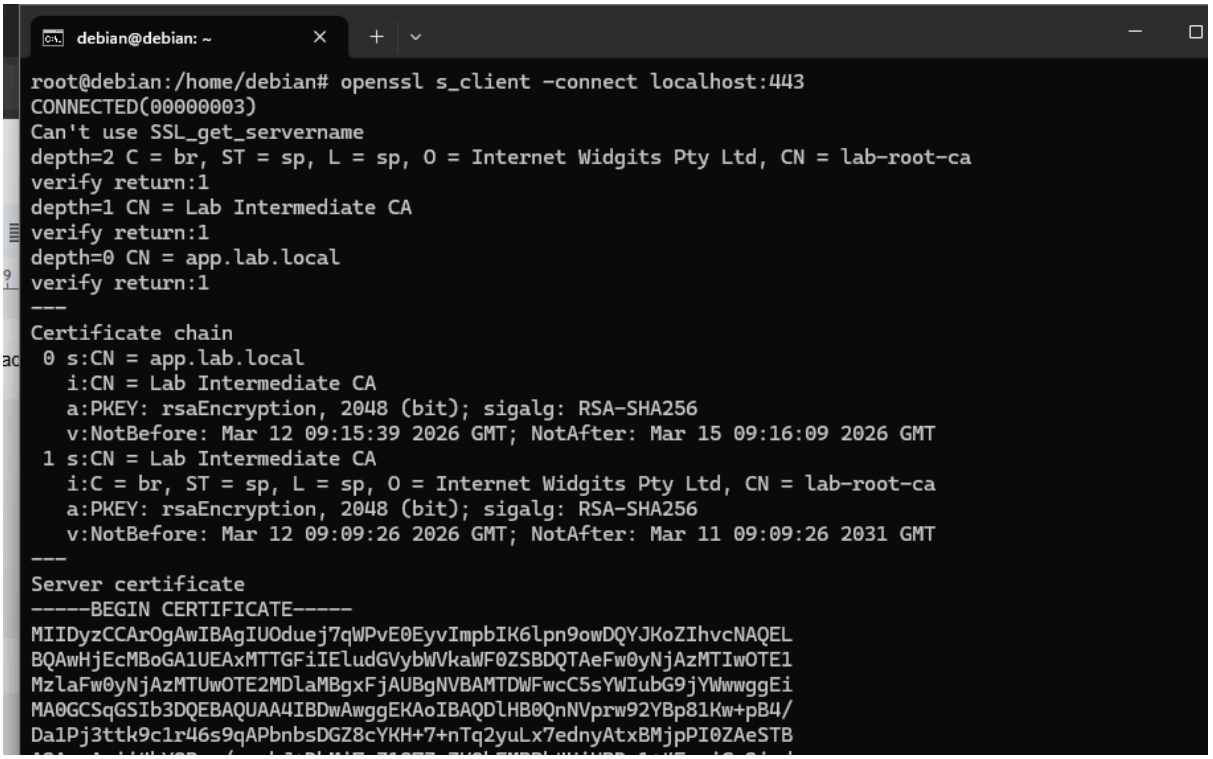

debian@debian: ~
2026-03-12T03:53:15.962-0500 [INFO] core: successful mount: namespace="" path=secret/ type=kv version="v0.25.0+builtin"
WARNING! dev mode is enabled! In this mode, Vault runs entirely in-memory
and starts unsealed with a single unseal key. The root token is already
authenticated to the CLI, so you can immediately begin using Vault.
You may need to set the following environment variables:
$ export VAULT_ADDR='http://127.0.0.1:8200'
The unseal key and root token are displayed below in case you want to
seal/unseal the Vault or re-authenticate.
Unseal Key: N9d/H/lCa+Tb3NMp1pfNVH1yLWJLm/q1mWmFc1pWpR0=
Root Token: hvs.vxWd0VIWD9TXawHIUb99gxt9
Development mode should NOT be used in production installations!
2026-03-12T03:57:06.470-0500 [INFO] secrets.pki.pki_9e789d5f: 9c9fbec5-2fb1-994d-2634-b79038874b9b: succeeded in migrating to issuer storage version 2
2026-03-12T03:57:06.470-0500 [INFO] core: successful mount: namespace="" path=pki_int/ type=pki version="v1.21.4+builtin.vault"
2026-03-12T03:57:15.815-0500 [INFO] core: mount tuning of leases successful: path=pki_int/
debian@debian: ~
updating /config/urls or the newly generated issuer with this information.
Since this certificate is an intermediate, it might be useful to regenerate
this certificate after fixing this problem for the root mount.
Key Value
--- -----
csr -----BEGIN CERTIFICATE REQUEST-----
MIICYzCCAUsCAQAwHjEcMBoGA1UEAxMTTGFiIEludGVybWVkaWF0ZSBDQTCCASIw
DQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAO0MbxaytJwpGniEKkUvXDUv5L6m
bEQ/BefPMwEBLavhUmBDMm56Oema6wrwzvpeKNqE9to94ziJhdF3xu/3Rw1gfASo
88X0DA2ZtuDeUrikRDGDCI7clIMyhLUcDXeJV84zdUKpcvQLl3jjCPeynAFyYyem
nRgufzBOZMTuWpF21iaC7RPrYcAOpE83/kXrzGhW1Ir2+A+15b3k+IvlsIFsLHCE
G3jKn9RbTHCRJ0pQnlDs8KNpF1Knb0JW05FItx6a7QZFiotc/pmwz914uw2gB+Du
KOJjApcB21LwGxkQ+SN6r8v9pzyRfYV22PrEQlYtCMBQOJnfTdd6m7GZUdUCAwEA
AaAAMA0GCSqGSIb3DQEBCwUAA4IBAQCzKYTGzA/iAx8lv5yrOA8HwpEia2uc/4Q7
b33zMd4XgkYxIJrWMtBZ7H8QWggsReeKmbk4FHxrG6n7r/PrgDCBL0lN1recdfJj
WeOA+EPm1xMCv9FJorsye67knhKT7SGEtWe6ZSZoTVlIJ3u5xiPL2rhcbOineMDX
admIcJX0SNKLIDTOspyUZv0FzZTnhN8/Mb3Y+9THV5g+I3f5hvB5UgSp1H+6lIXb
INvNoy4HAMKfFkDMmAdP/ydmBErJvoMFY2PxVZ0MWwhfnQw7FJfP0bckuSNSL+wB
9IKQpFr7UaNsAkn9aqy6ienGarjrwjJPFHVq8CFTPqf4sloxZyQh
-----END CERTIFICATE REQUEST-----
key_id d09b8923-766f-6457-4222-9fbb02074f2d



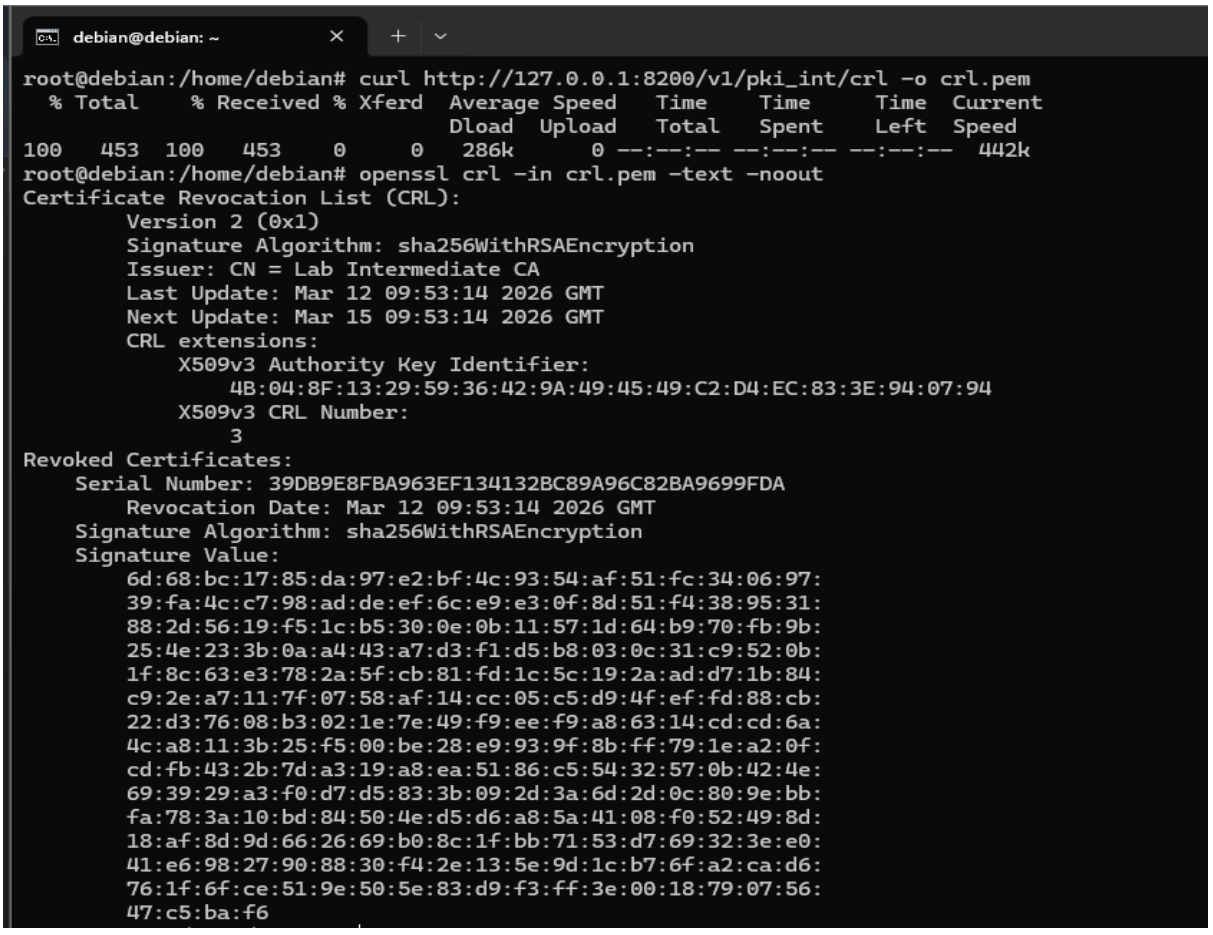
Configurando a intermediate.

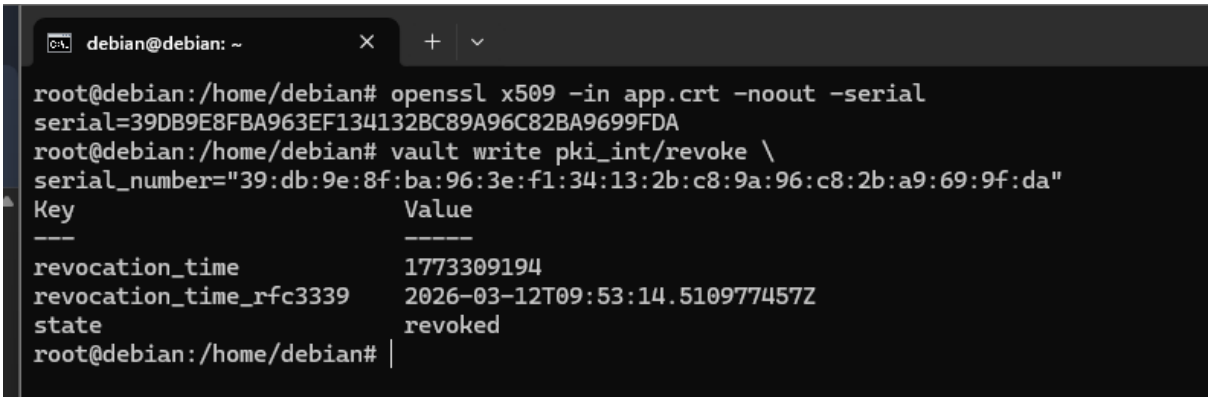


Substituindo os certificados do apache.

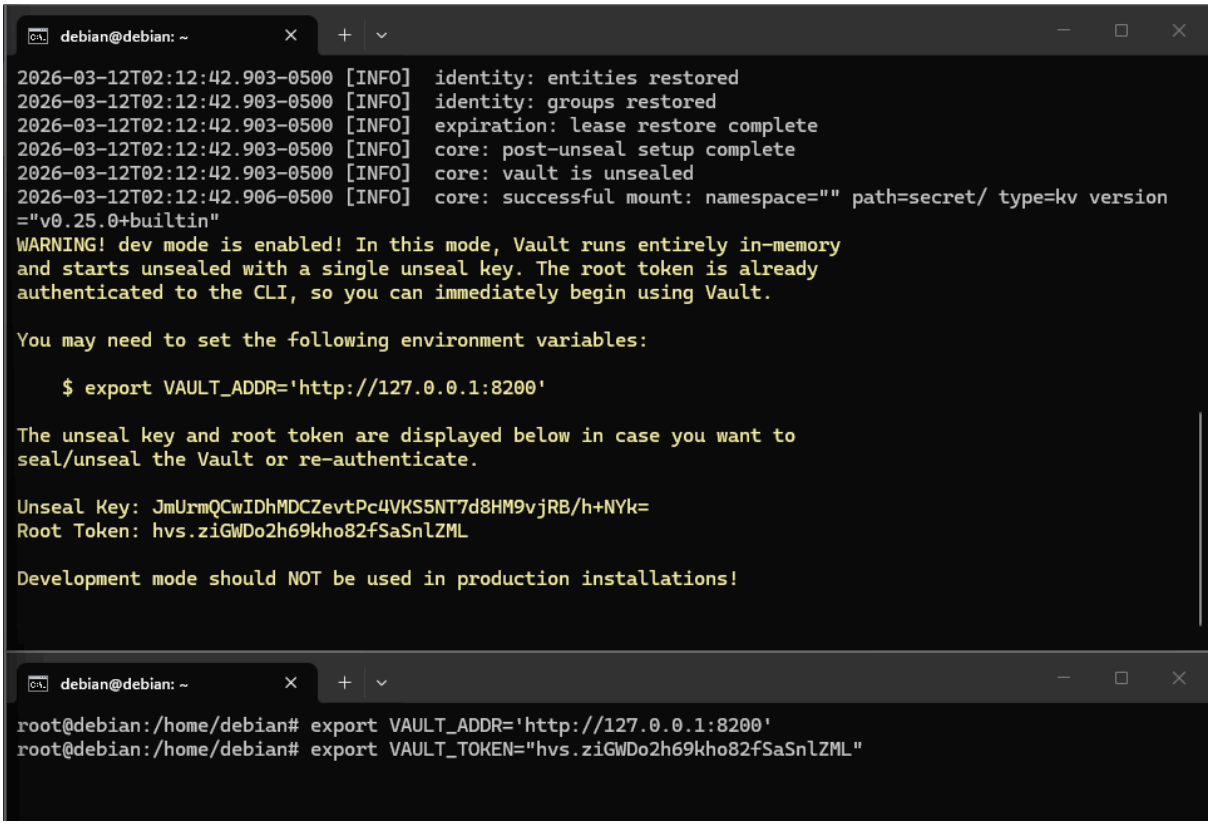


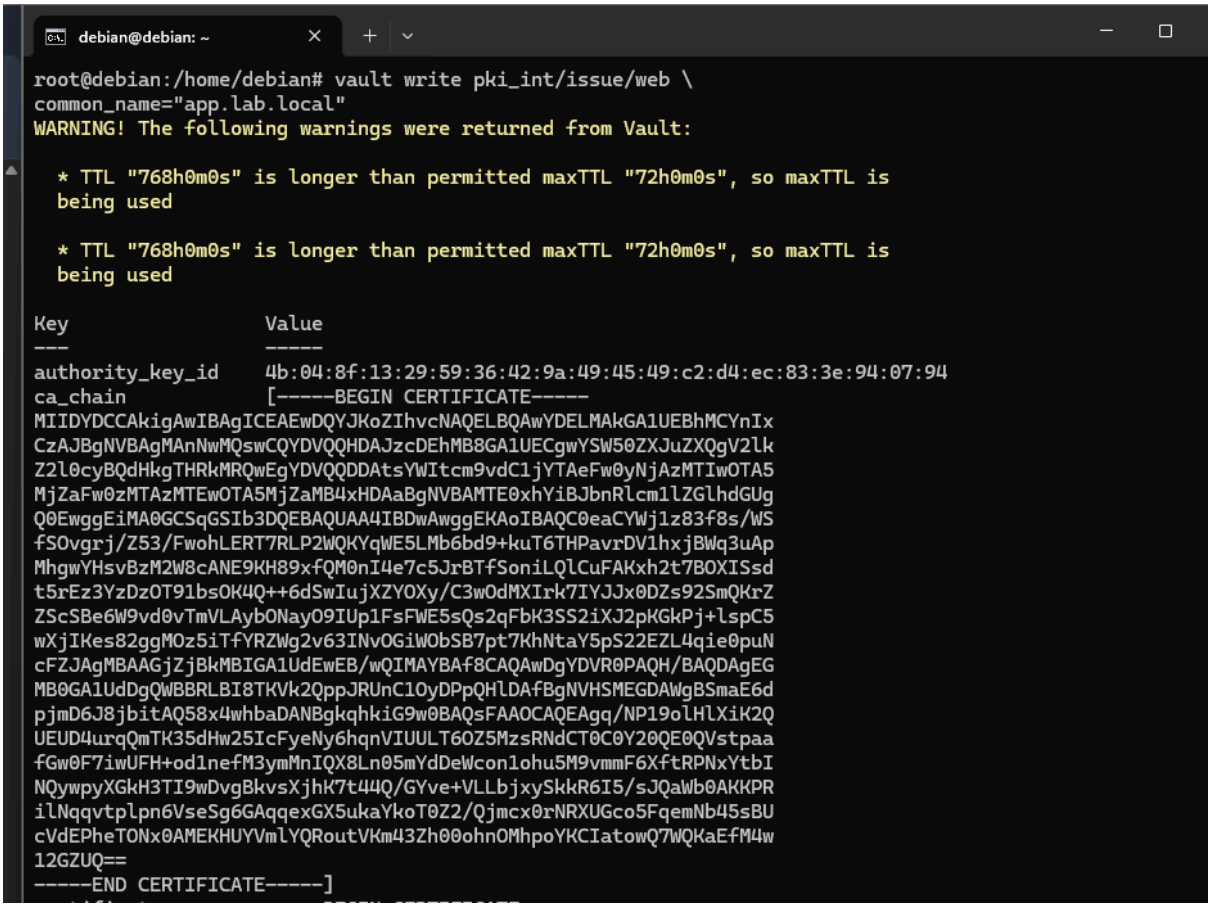
Novo certificado aplicado no apache.





Revogando certificado.





Gerando certificado.

7. Conclusão:

Ao final deste laboratório, foi possível compreender e implementar os principais componentes de uma infraestrutura de certificação digital. A utilização do SoftHSM permitiu simular o funcionamento de um HSM, garantindo que a chave privada da Autoridade Certificadora fosse armazenada e utilizada de forma segura por meio do padrão PKCS#11. Com o auxílio do OpenSSL, foi possível gerar solicitações de certificado, criar uma CA e assinar certificados para uso em um servidor web seguro configurado no Apache HTTP Server.

Além disso, a introdução do HashiCorp Vault demonstrou como ferramentas modernas podem ser utilizadas para centralizar e automatizar o gerenciamento de segredos e certificados, aumentando a segurança e reduzindo a complexidade operacional em ambientes de produção. Dessa forma, o laboratório permitiu consolidar conhecimentos fundamentais sobre PKI, proteção de chaves criptográficas e gestão de certificados digitais, conceitos amplamente utilizados em infraestruturas corporativas, serviços em nuvem e arquiteturas de segurança modernas.